Deep Learning

Lecture 7

Daniil Dorin



Experience

АНТИПЛАГИАТ

Senior CV Researcher
Developer

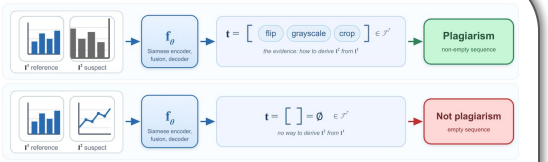
Education

- Bachelor: MIPT with honours
- Master: MIPT with honours
- PhD student: MIPT

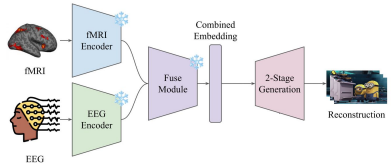
Past Experience

Technician

Projects



Evidential Image Matching: Predicting Transformation Sequences to Derive One Image from Another



Decoding Visual Information from Neural Signals: Image Reconstruction Based on Joint fMRI and EEG Analysis

Recap

- Language Modeling
  - RNN
  - LSTM
  - GRU

- Machine Translation
  - RNN with Attention
  - Transformer architecture

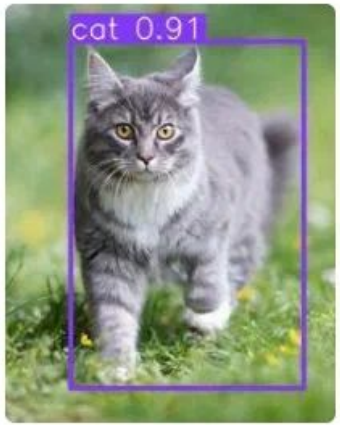
- Large Language Models
  - Fine-tuning, LoRA
  - RAG
  - Agents

Classical computer vision tasks

Seminar

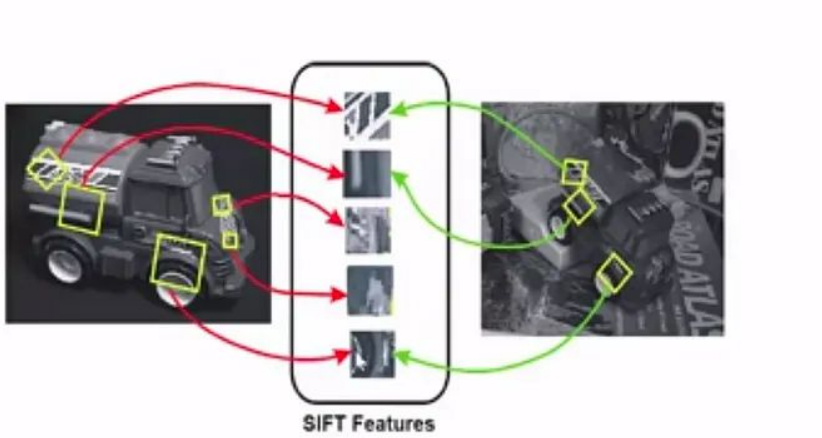
Lecture

Next Lecture

Classification	Detection	Segmentation
		

Classical approach: SIFT vectors

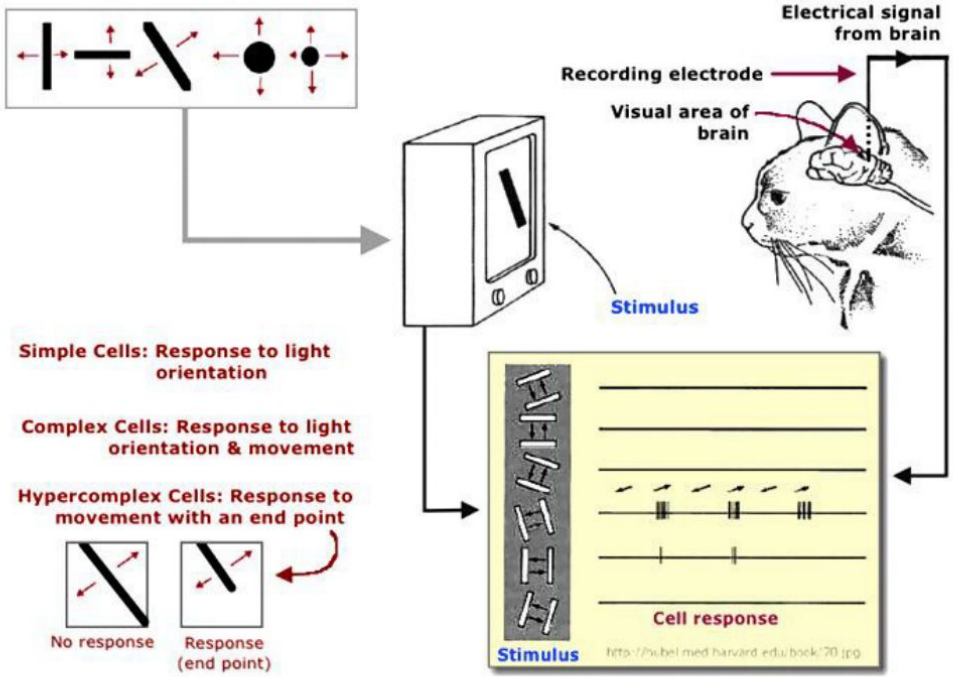
We should somehow extract features from images to build a classification model or match images.



Question: How can we do it?

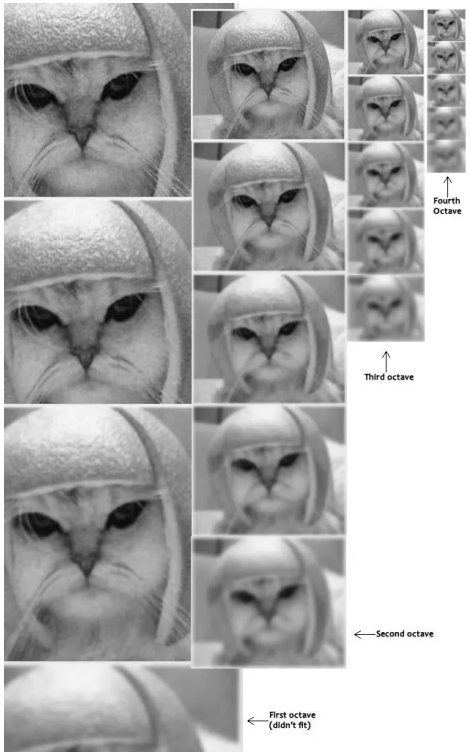
Distinctive Image Features from Scale-Invariant Keypoints (2004)

Why edges are important?



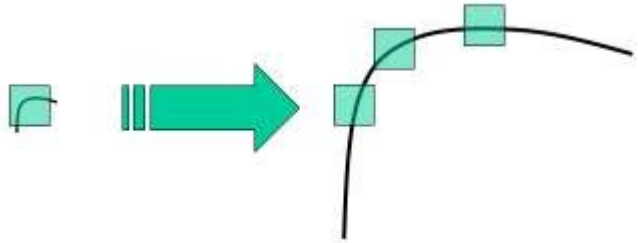
Hubel & Wiesel, 1959

SIFT vectors



Real world objects are meaningful only at a certain scale. You might see a sugar cube perfectly on a table. But if looking at the entire milky way, then it simply does not exist.

This multi-scale nature of objects is quite common in nature. And a scale space attempts to replicate this concept on digital images.



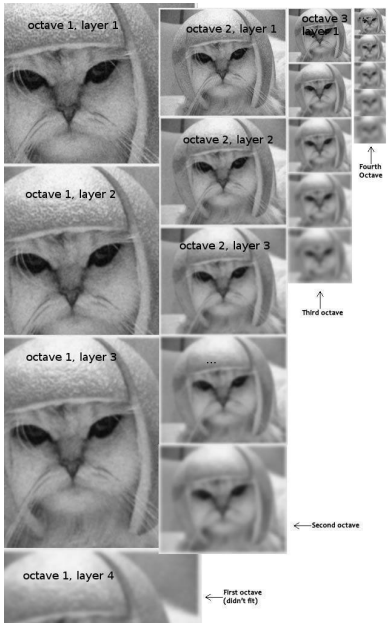
The basic concept of the image is edge, let's start by extracting edges

https://www.ultralytics.com/ru/blog/what-is-the-scale-invariant-feature-transform-sift
https://blog.roboflow.com/sift/

Classical approach: SIFT vectors



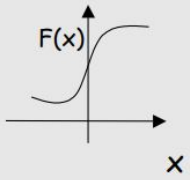
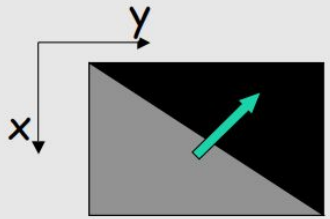
Using gaussian kernel to remove noise.



Then, repeat the operations on lower scale

Gaussian filter: $G(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$, **convolution:** $I'(i,j) = \sum_{k,l} I(i+k, j+l)\, G(k,l)$

Edge Detection Summary

	1D	2D
step edge	$I(x)$ 	$I(x,y)$ 
1st deriv	$\left\| \frac{dI(x)}{dx} \right\| > Th$	$\|\nabla I(x,y)\| = (I_x^2(x,y) + I_y^2(x,y))^{1/2} > Th$ $\tan\theta = I_x(x,y) / I_y(x,y)$
2nd deriv	$\frac{d^2I(x)}{dx^2} = 0$	$\nabla^2 I(x,y) = I_{xx}(x,y) + I_{yy}(x,y) = 0$ Laplacian

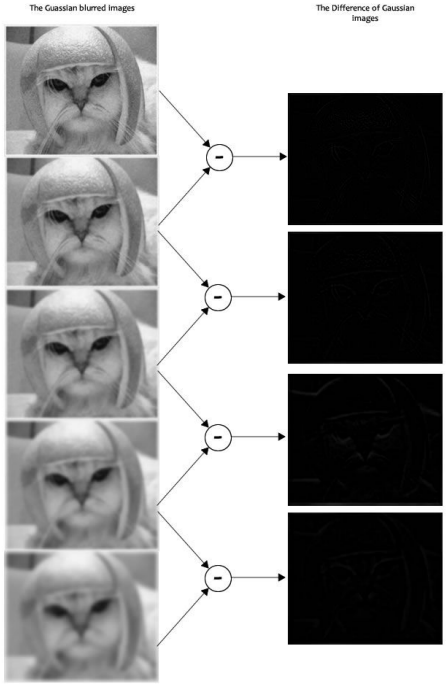
Laplacian of Gaussian (LoG)

Laplacian of Gaussian locates edges and corners on the image. These edges and corners are good for finding keypoints.

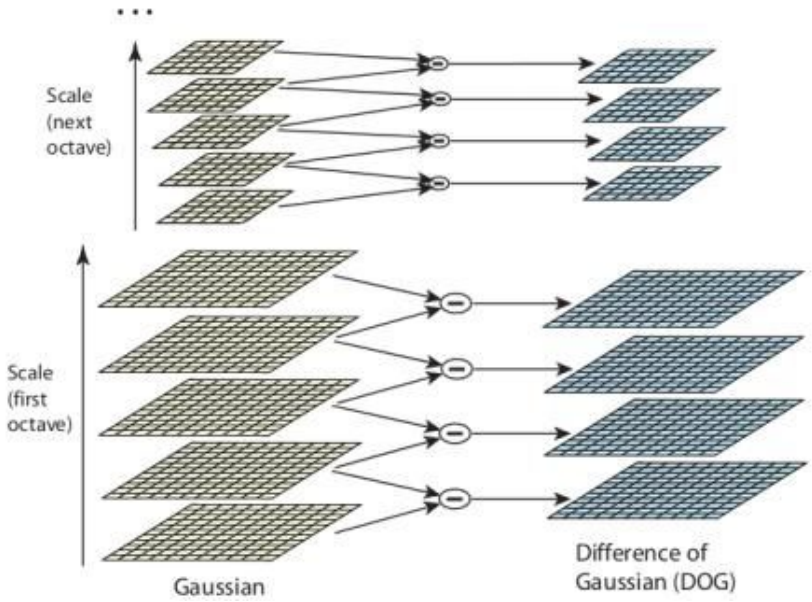


But it is computationally hard calculate. One can use simple approximation using **difference of Gaussians.**

LoG result



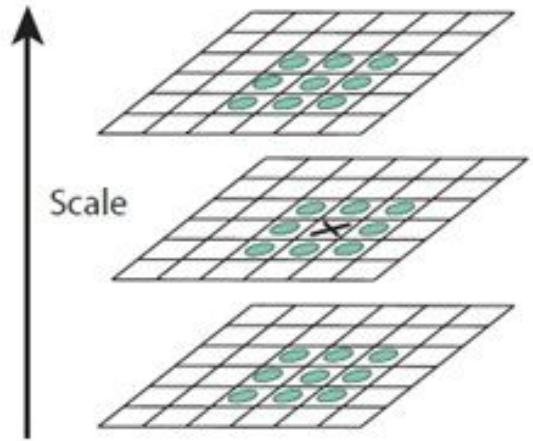
What we will get?

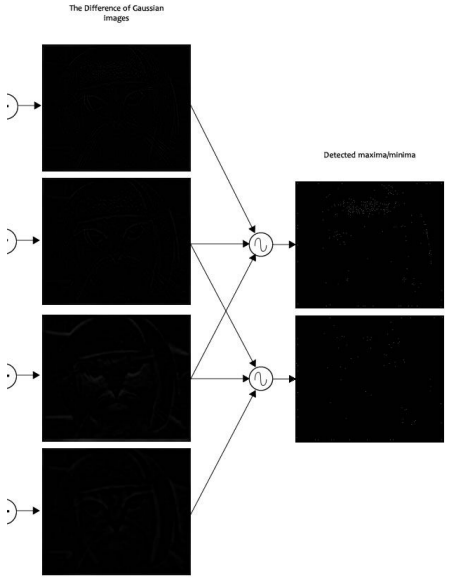


Repeating procedure over scales

Finding keypoints

Next, to find keypoints we are looking for maximum and minimum. The author propose the following scheme: we compare the pixel marked with cross with all his 26 neighbours (left picture).





As we got 4 DoG images in each octave, we can obtain 2 images with keypoints.

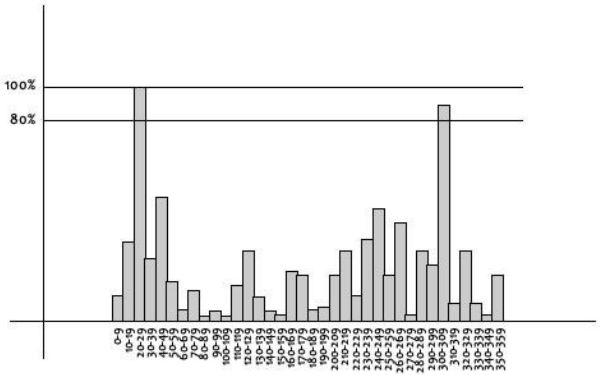
Orientation Assignment

The next thing is to assign an orientation to each keypoint. This orientation provides rotation invariance. The more invariance you have the better it is. Actually, it is very important when we do matching between different images. The same object can have different orientation. To calculate the orientation we use the following formulas:

$$m(x,y) = \sqrt{(L(x+1,y) - L(x-1,y))^2 + (L(x,y+1) - L(x,y-1))^2}$$

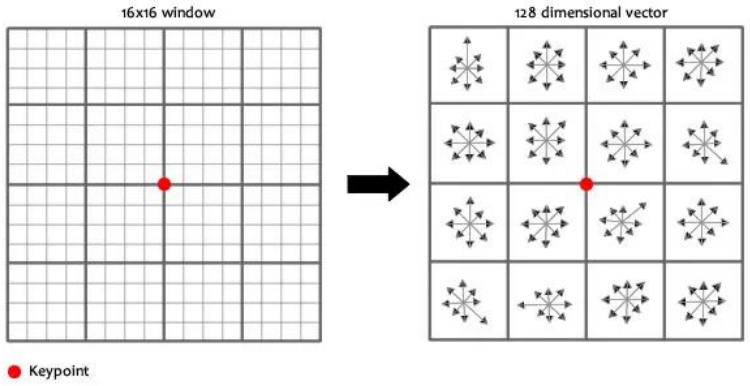
$$\theta(x,y) = \tan^{-1}((L(x,y+1) - L(x,y-1))/(L(x+1,y) - L(x-1,y)))$$

Then, we create a histogram with magnitude distribution. Any other peak higher than 80% of the maximum gives a new keypoint with that orientation.

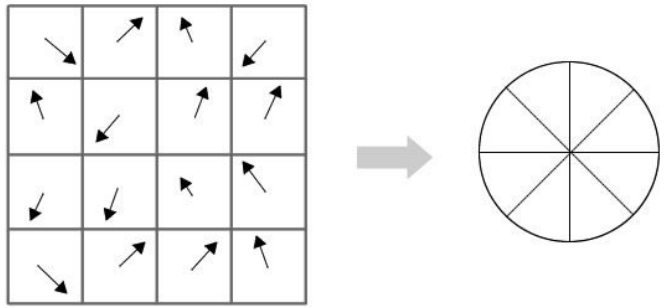


Keypoint descriptor

We want to generate a very unique fingerprint for the keypoint. It should be easy to calculate. We also want it to be relatively lenient when it is being compared against other keypoints. To do this, a 16x16 window around the keypoint. This 16x16 window is broken into sixteen 4x4 windows.

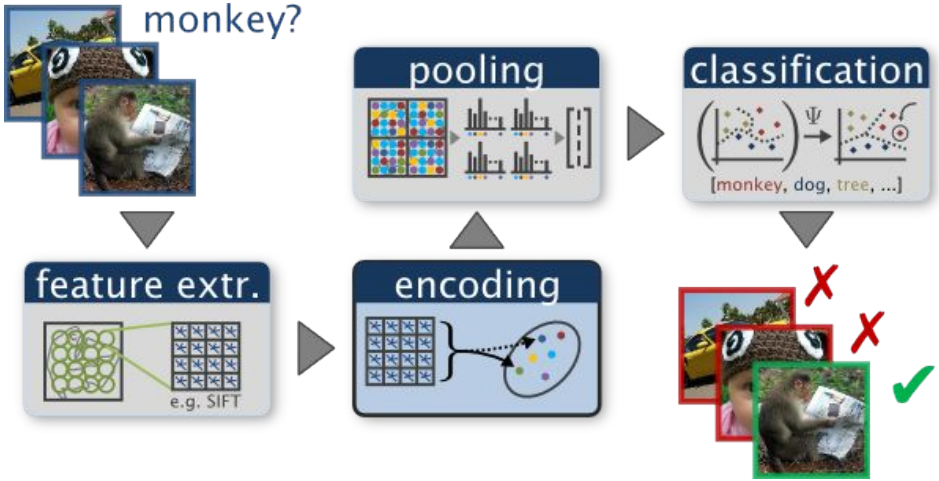


First, we create 16x16 window around the keypoint



Then, inside a window we calculate distribution of gradients = 8 features

Classical solutions



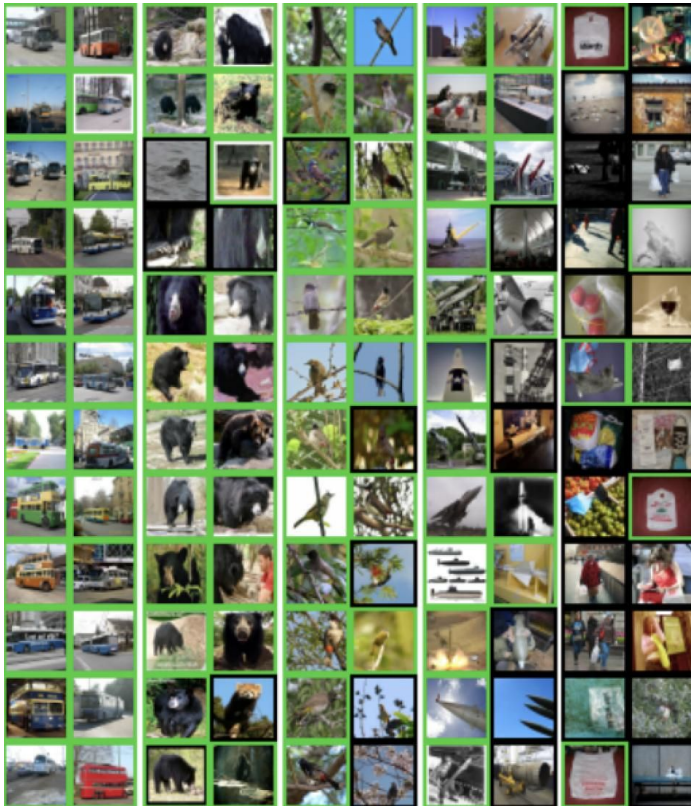
Before deep learning era solution for classification task has the following form:

1. Detect SIFT keypoints: find stable local features across scales and rotations.
2. Extract 128-D descriptors for each keypoint.
3. Cluster descriptors (K-means): group all descriptors into clusters → form a visual vocabulary ("visual words").
4. Build Bag-of-Words histogram: for each image count how many descriptors fall into each cluster.
5. Train SVM on histograms: use histograms as feature vectors to classify images.

Neural network solutions

ImageNet

IMAGENET



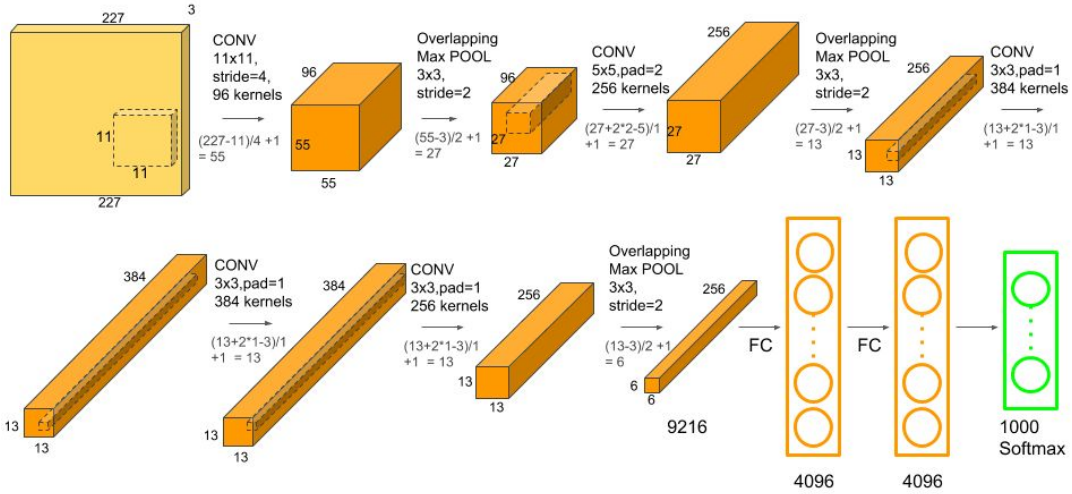
“Everything in deep learning starts with data”

DL in Computer Vision started with ImageNet:

- ImageNet is a dataset of 1.2 M / 50k / 100k images (train/val/test)
- Consists of 1000 different image classes.

ImageNet: A Large-Scale Hierarchical Image Database (2009)

AlexNet



Architecture AlexNet

Imagenet classification with deep convolutional neural networks (2012)

AlexNet: success factors

- Replaced tanh (LeNet-5) with ReLU (x6 speedup)
- Dropout + Augmentations
- 5 conv layers (11x11,5x5,3x3,3x3,3x3)
  LeNet-5 has 2 conv layers
- Test-time augmentations (5 crops x horizontal flip)

Local normalization (analogue to batch normalization)

$$b^i_{x,y} = a^i_{x,y} / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2 \right)^{\beta}$$

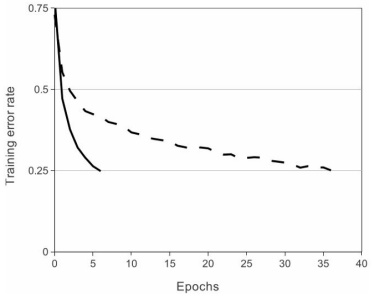
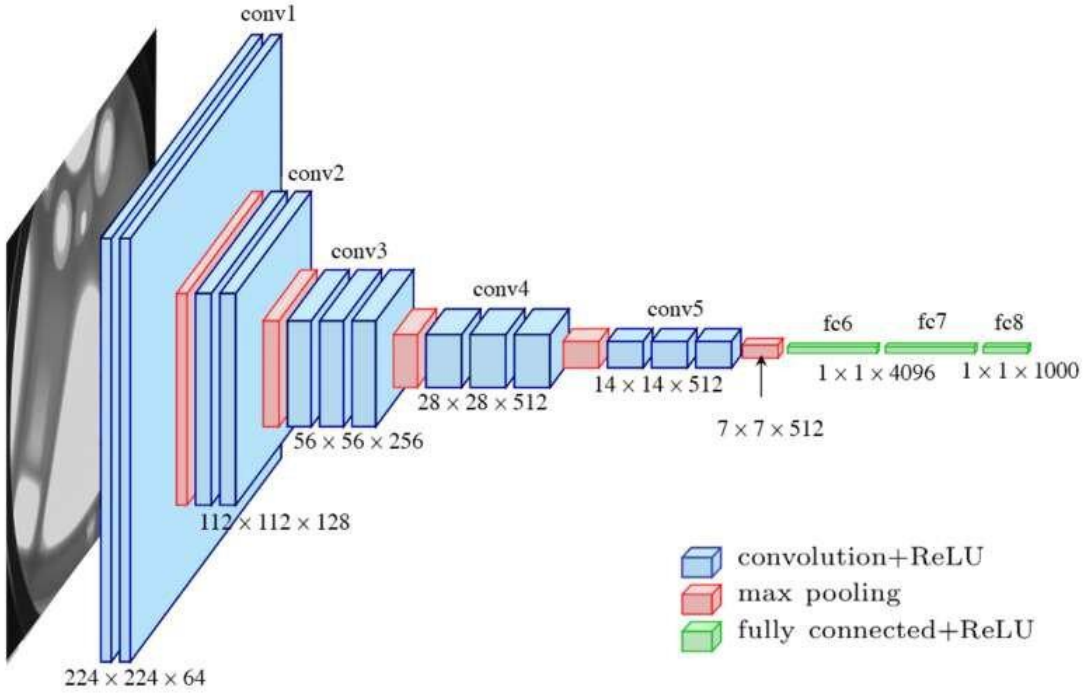


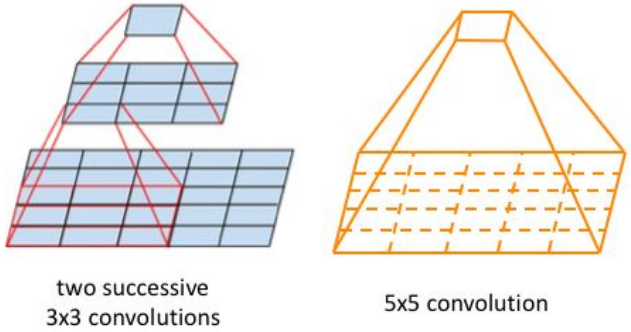
Figure 1: A four-layer convolutional neural network with ReLUs **(solid line)** reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with tanh neurons **(dashed line)**. The learning rates for each network were chosen independently to make training as fast as possible. No regularization of any kind was employed. The magnitude of the effect demonstrated here varies with network architecture, but networks with ReLUs consistently learn several times faster than equivalents with saturating neurons.

VGG



Very deep convolutional networks for large-scale image recognition (2015)

VGG: cascade of kernels



Instead of using, 5x5 conv authors proposed to use two 3x3 conv
It’s computationally more efficient (20 params vs 26 params), but the receptive field is equal

VGG: stagewise training

Initial architecture

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

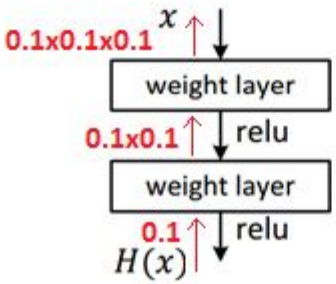
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

The final architecture

Instead of training all architecture end-to-end, they created a simple versions of architectures, trained them and added additional layers after the previous stage was completed.

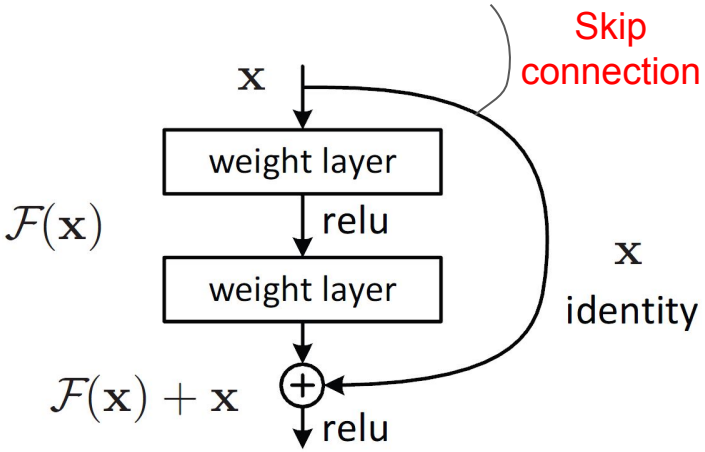
Vanishing gradient problem

Let’s draw a little bit

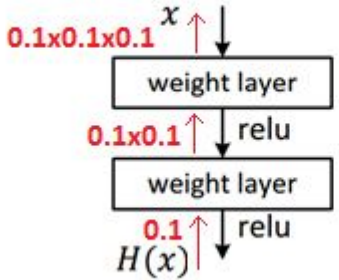


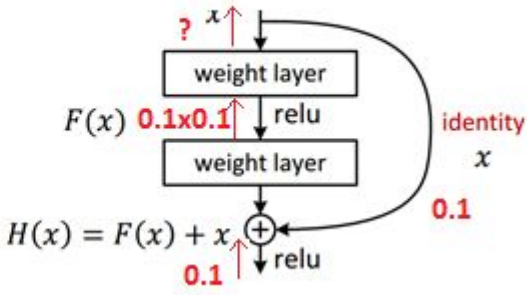
$$\frac{\partial L}{\partial w_1} = \underbrace{\frac{\partial L}{\partial H}}_{\text{grad from output}} \cdot \underbrace{\frac{\partial H}{\partial h_2}}_{\text{grad through layers}} \cdot \underbrace{\frac{\partial h_2}{\partial w_1}}_{\text{grad to weight}}$$

Skip connection



To prevent model from vanishing gradient problem one can use **skip connections**

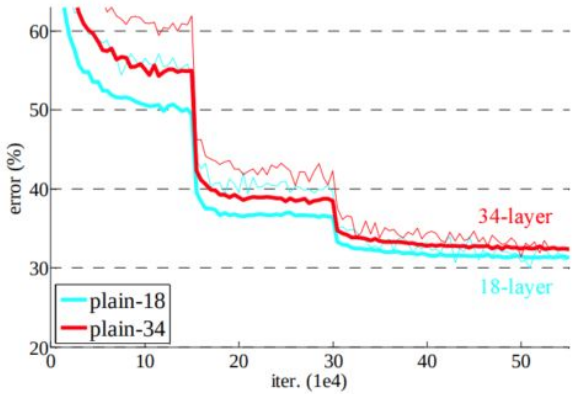


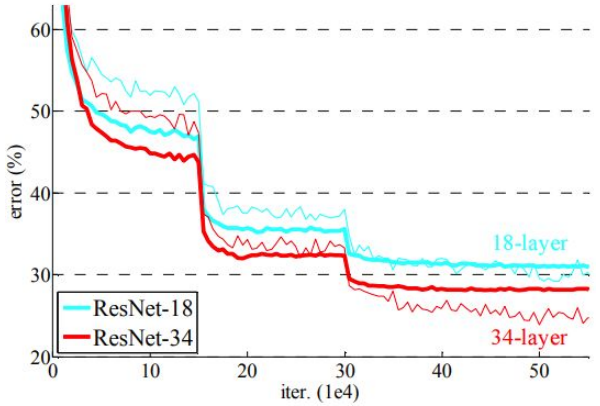


Creating highway allows to keep the gradient non-saturating + allows to pass information from bottom layer to top.

ResNet

For plain networks, the deeper model is even worse.

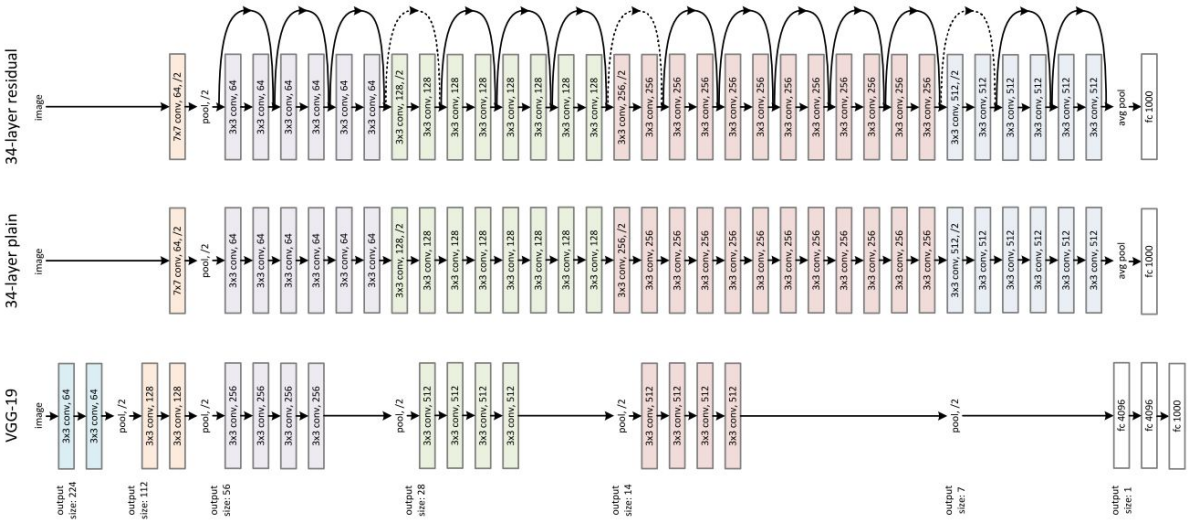




Results for the deeper model are better! Success!

Deep Residual Learning for Image Recognition (2015)

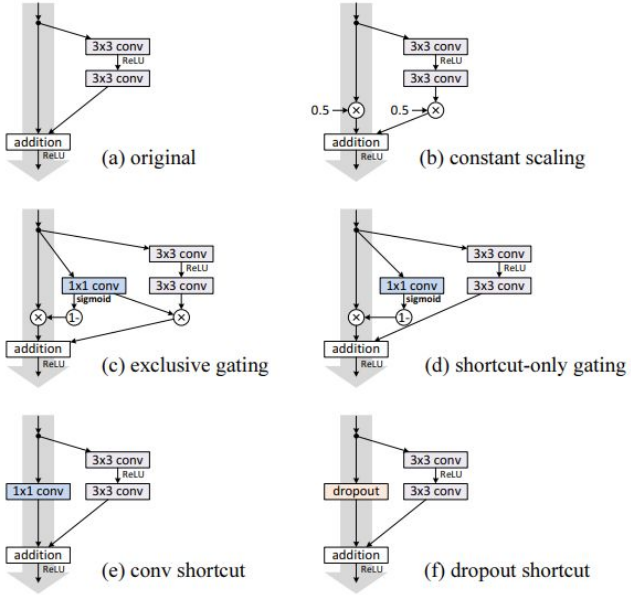
ResNet

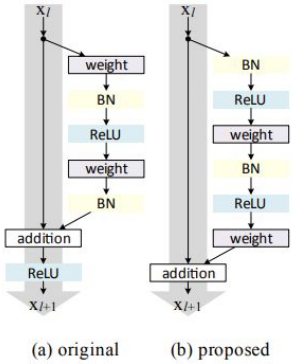

VGG-19
34-layer plain
34-layer residual
image
3x3 conv, 64
3x3 conv, 128
3x3 conv, 256
3x3 conv, 512
7x7 conv, 64, /2
3x3 conv, 128, /2
3x3 conv, 256, /2
3x3 conv, 512, /2
pool, /2
avg pool
fc 4096
fc 1000
output size: 224
output size: 112
output size: 56
output size: 28
output size: 14
output size: 7
output size: 1

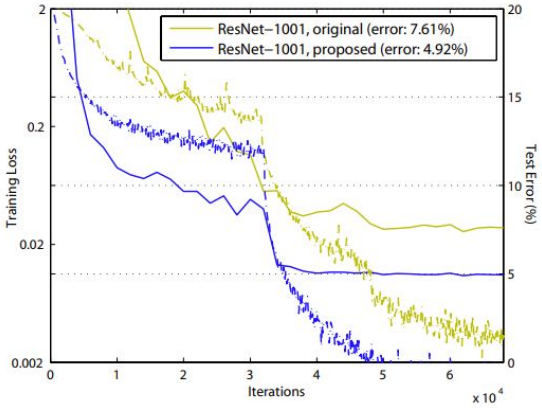
ResNet

Identity Mappings in Deep Residual Networks (2016)

What is the optimal residual layer form?





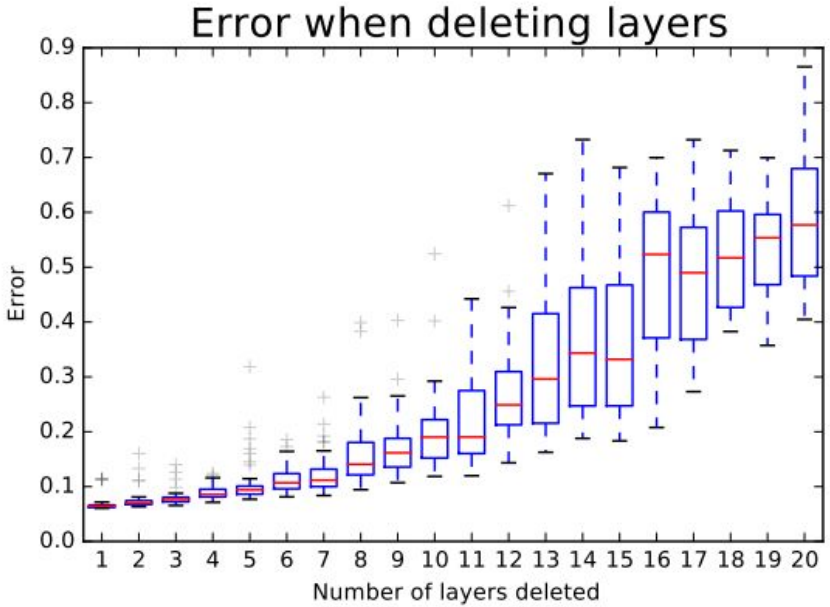


The best strategy for residual connection is $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l)$

ResNet: cool feature

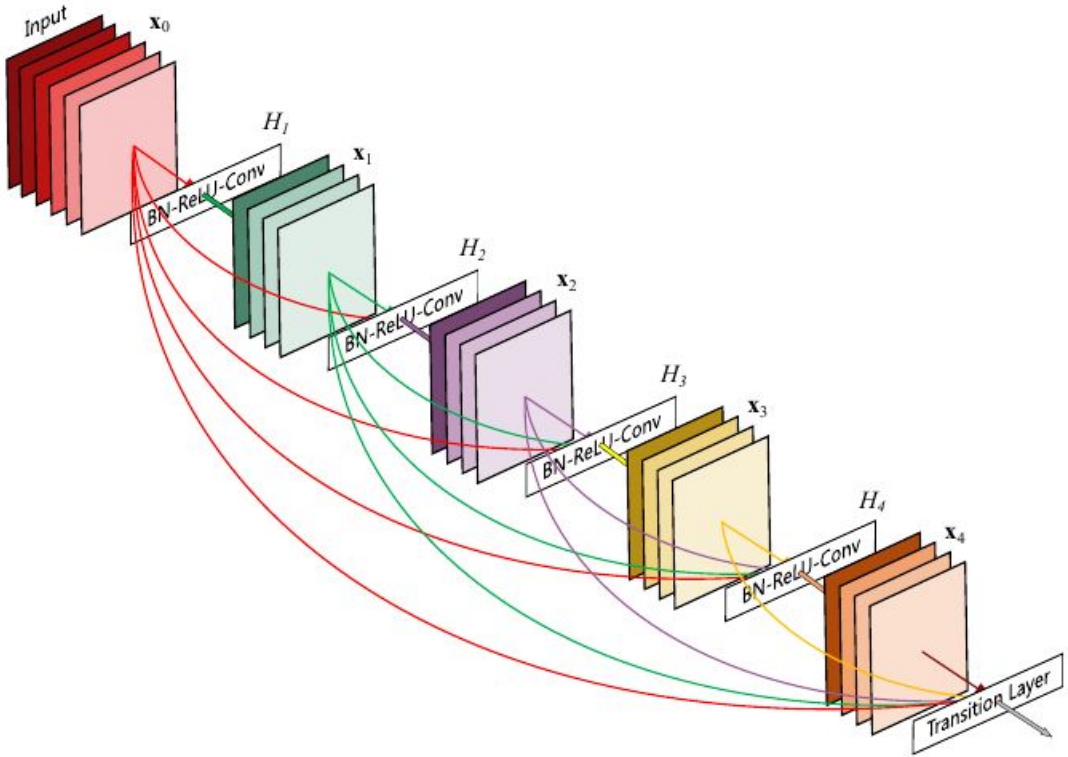
Interesting fact: ResNet can be a few layers out of ResNet and its performance wouldn't decrease too much.

This is achieved by the fact that ResNet has "workarounds" for information (skip-connection), and if one of them is interrupted. one of them, the information can still can still pass through the other



Residual Networks Behave Like Ensembles of Relatively Shallow Networks (2016)

DenseNet

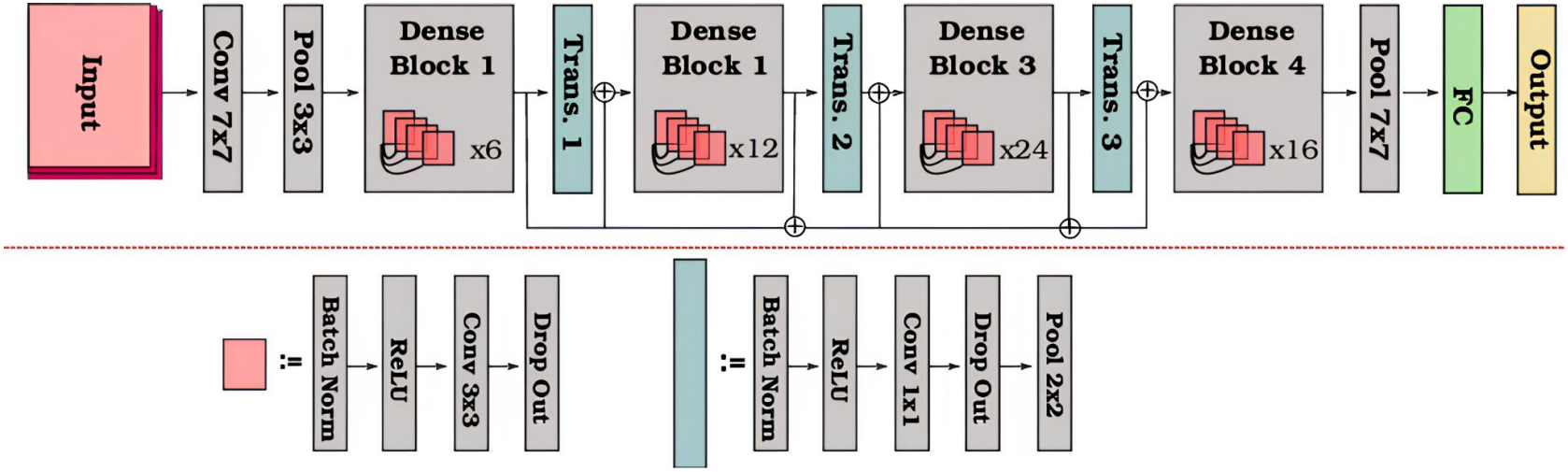


What if each successive layer of the network will receive as input the outputs of all previous layers (instead of only one in ResNet)

DenseNet

Densely Connected Convolutional Networks (2017)

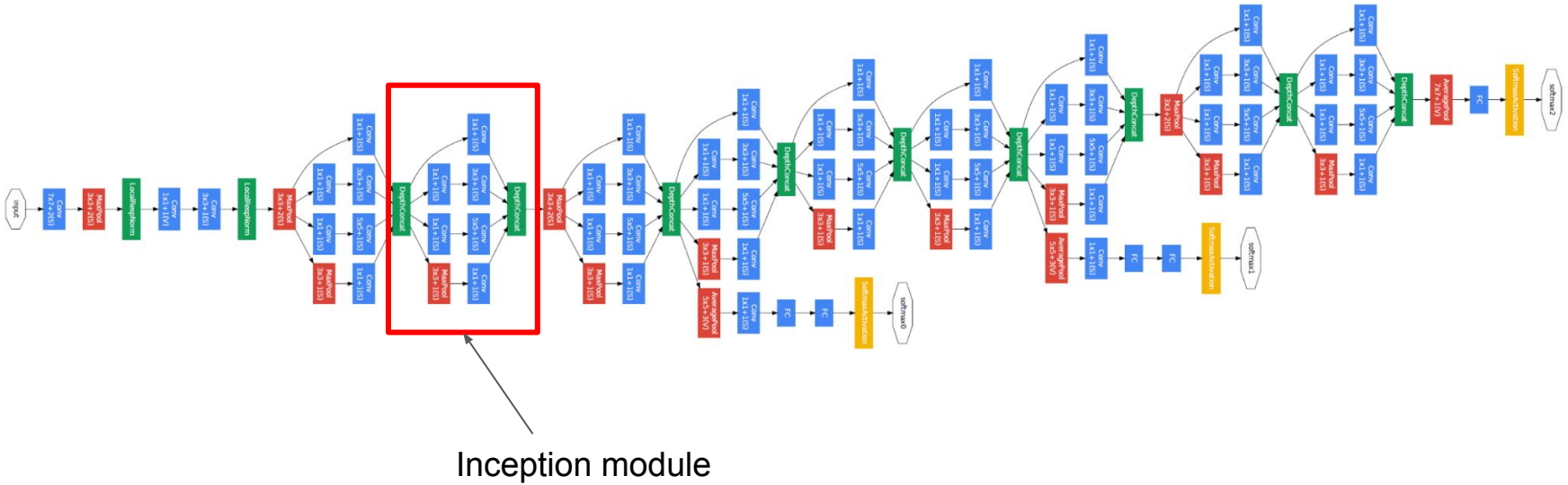
DenseNet


Input
Conv 7x7
Pool 3x3
Dense Block 1
x6
Trans. 1
Dense Block 1
x12
Trans. 2
Dense Block 3
x24
Trans. 3
Dense Block 4
x16
Pool 7x7
FC
Output
Batch Norm
ReLU
Conv 3x3
Drop Out
Batch Norm
ReLU
Conv 1x1
Drop Out
Pool 2x2

DenseNet

- Strong gradient flow
- The number of layers and parameters is not very large
- Conv layers emphasise a wider variety of features
- Higher conv layers take into account low-complex patterns from lower layers, which can be useful for detecting some low-level patterns.

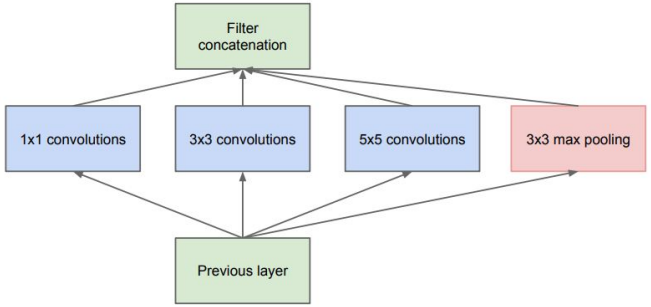
Inception | GoogLeNet



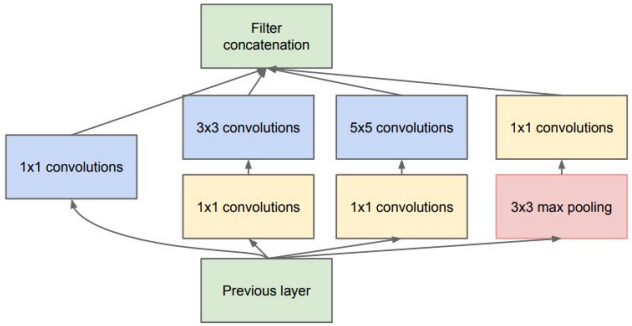
- 22 layer
- Additional outputs for classification

Going Deeper with Convolutions (2014)

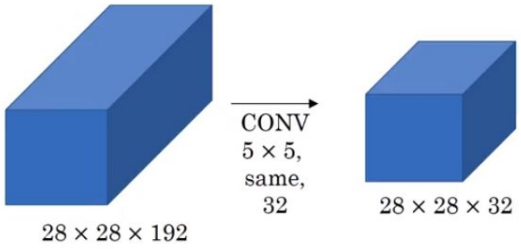
Inception module



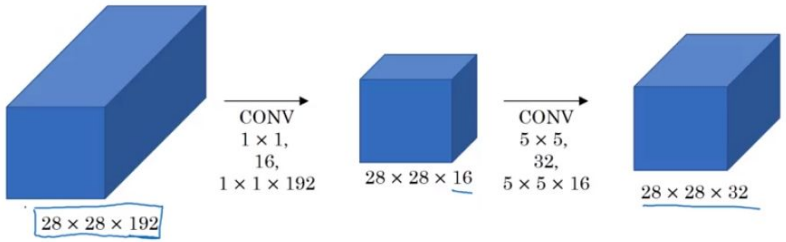
(a) Inception module, naïve version



(b) Inception module with dimension reductions



$\approx 120M$ calculations

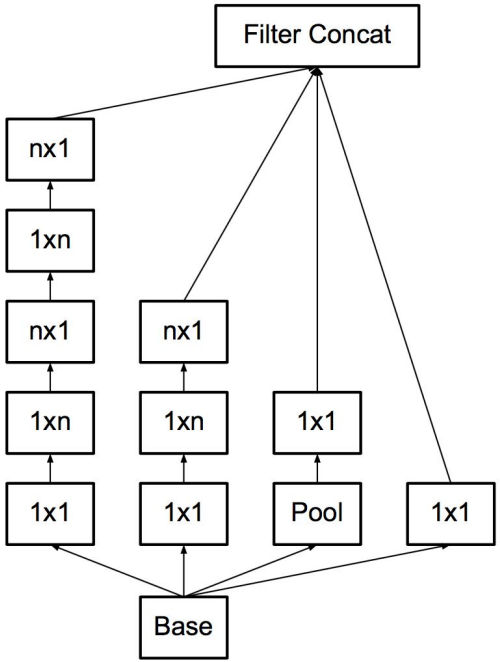


$\approx 12.4M$ calculations

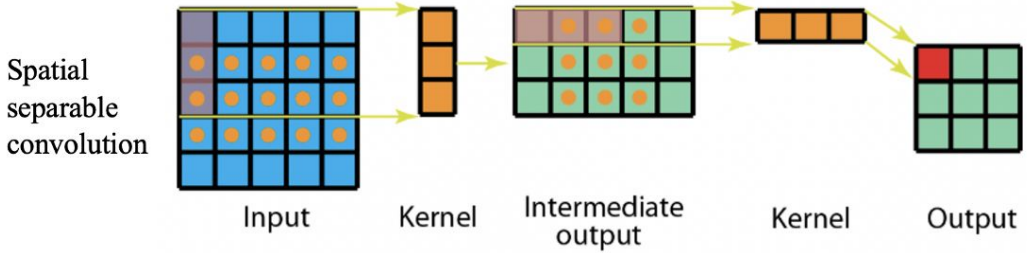
$$\text{FLOPs} = H_{out} \times W_{out} \times C_{out} \times K_h \times K_w \times C_{in}$$

ten times less calculations!

Inception v2, v3

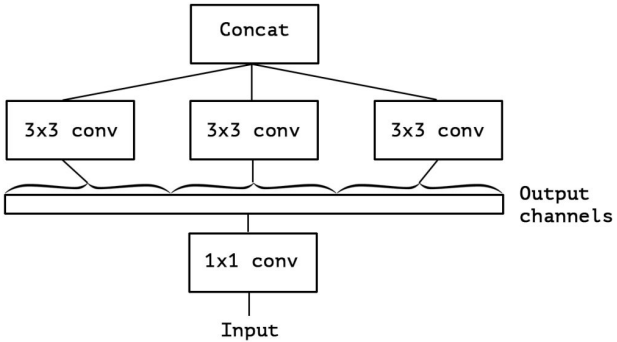


How to represent 3x3 convolution by composition of two one dimensional convolutions?

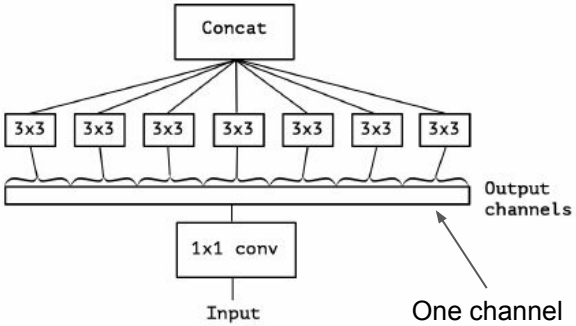


Rethinking the Inception Architecture for Computer Vision (2016)

Xception



Standardized Inception module form

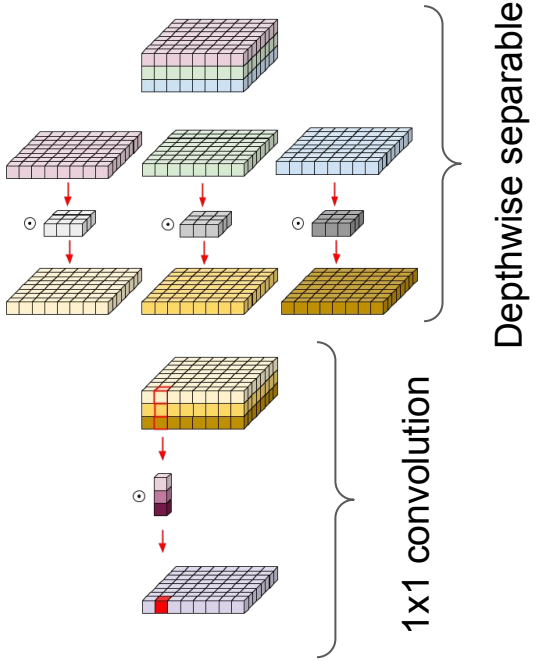


Xception module

Hypothesis: cross-channel correlations and spatial correlations are sufficiently decoupled that it is preferable not to map them jointly

Xception: Deep learning with depthwise separable convolutions (2016)

Xception



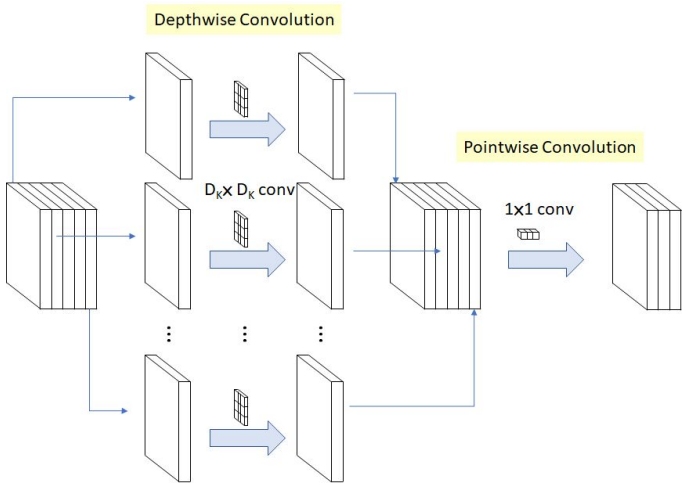
How Xception architecture looks like?

Depthwise separable + 1x1 conv
Add residual connections

Table 1. Classification performance comparison on ImageNet (single crop, single model). VGG-16 and ResNet-152 numbers are only included as a reminder. The version of Inception V3 being benchmarked does not include the auxiliary tower.

	Top-1 accuracy	Top-5 accuracy
VGG-16	0.715	0.901
ResNet-152	0.770	0.933
Inception V3	0.782	0.941
Xception	**0.790**	**0.945**

MobileConv



Convolution calculations:

$$\text{FLOPs}_{\text{full}} = K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W$$

Xception calculations:

$$\text{FLOPs}_{\text{sep}} = K^2 \cdot C_{\text{in}} \cdot H \cdot W + C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W$$

$$\frac{\text{FLOPs}_{\text{sep}}}{\text{FLOPs}_{\text{full}}} = \frac{K^2 \cdot C_{\text{in}} \cdot H \cdot W + C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W}{K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W}$$

$$= \frac{1}{C_{\text{out}}} + \frac{1}{K^2}$$

If we take 512 out channels and kernel size K=3 → we get approximately 9 times less calculations

MobileNet

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Got rid of all the overhead (no residual connections); the goal is not to match Xception accuracy, but to achieve an extremely lightweight model.

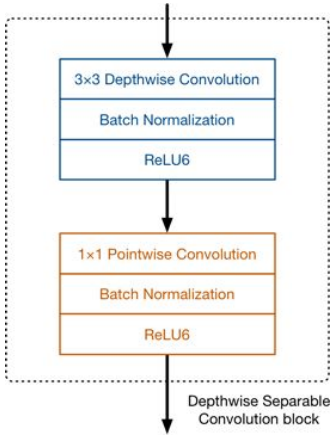
Table 8. MobileNet Comparison to Popular Models

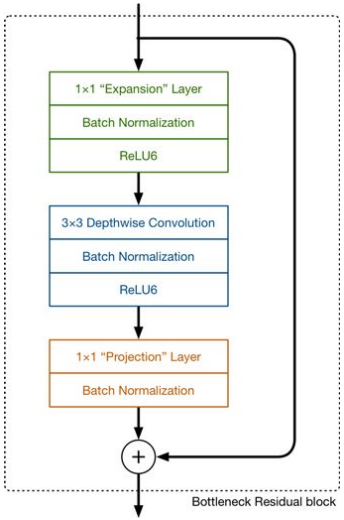
Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

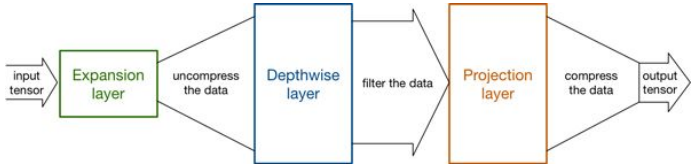
Same quality as GoogLeNet with ~2.7 times fewer calculations and ~1.6 times fewer parameters

MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications (2017)

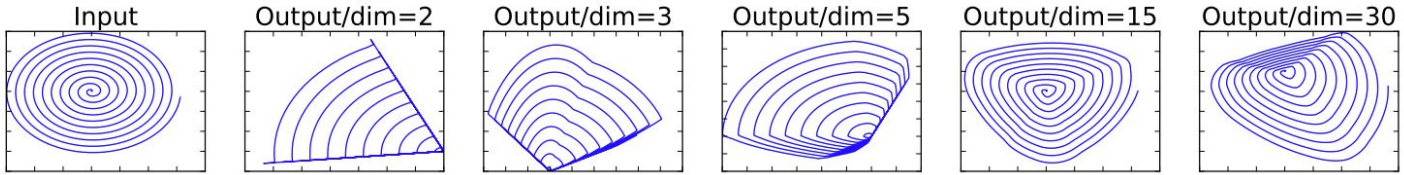
MobileNetV2





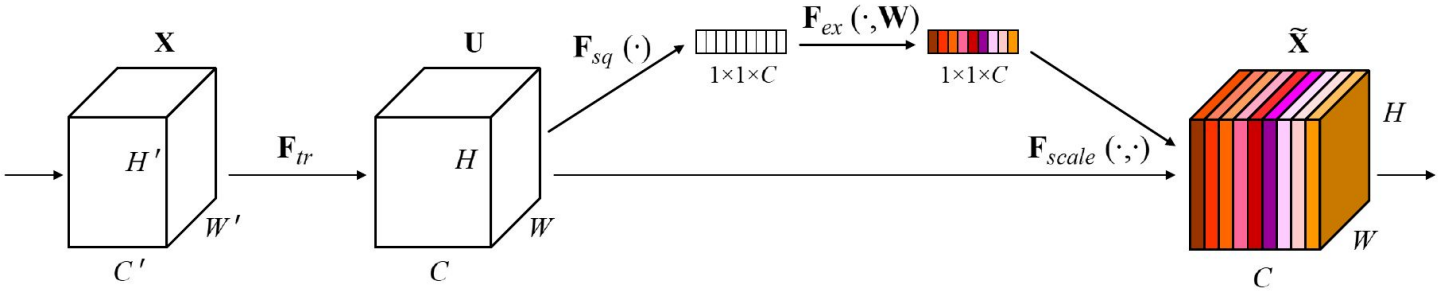


ReLU in low dimension can kill a lot of information



MobileNetV2: Inverted Residuals and Linear Bottlenecks (2018)

Squeeze-and-Excitation (SE) block

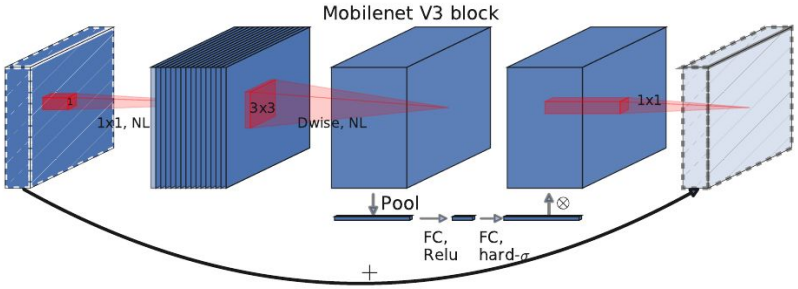


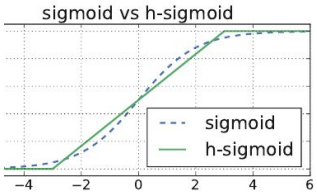
$s = \sigma(W_2 \cdot \mathrm{ReLU}(W_1 \cdot \mathrm{avgpool}(U)))$ $\quad \tilde{X}_c = s_c \cdot U_c$

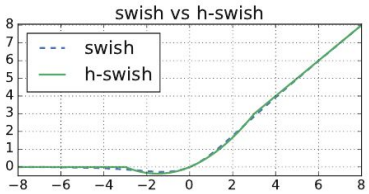
- **Squeeze:** global average pooling gives one number per channel
- **Excitation:** FC → ReLU → FC → sigmoid with a C/r bottleneck gives a weight in (0, 1) per channel
- **Scale:** each channel is multiplied by its weight, i.e. attention over channels, not positions
- **SE-ResNet-50:** top-1 error 24.8% → 23.3% vs ResNet-50 at +0.26% FLOPs

Squeeze-and-Excitation Networks (2018)

MobileNetV3





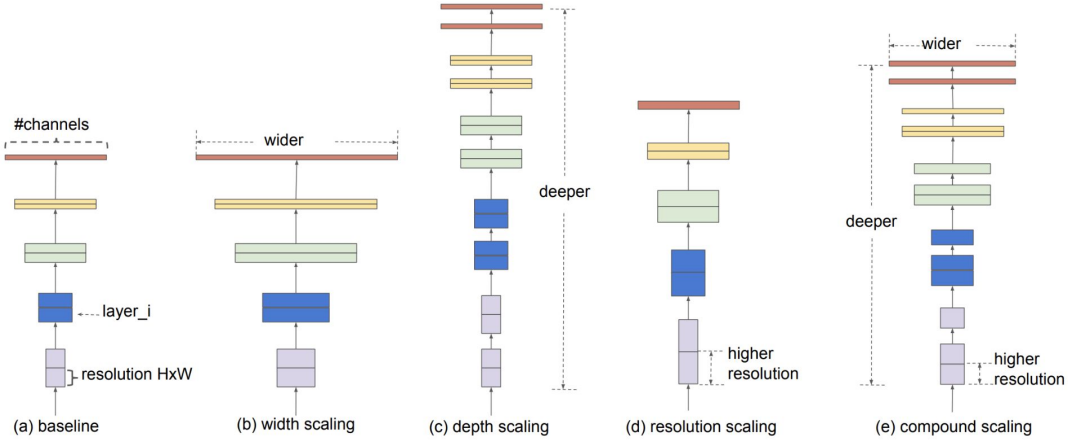


- **SE block** inside the inverted residual, after the depthwise conv; its bottleneck is 1/4 of the expansion channels
- **h-swish(x) = x · ReLU6(x + 3) / 6:** cheap piecewise-linear swish, the same trick gives hard-sigmoid in SE
- **Platform-aware NAS + NetAdapt** search the architecture for real phone latency
- **New last stage:** −7 ms (11% of runtime) and −30M MAdds at almost the same accuracy
- **V3-Large vs V2:** 75.2% vs 72.0% top-1, 51 vs 64 ms on Pixel 1

Searching for MobileNetV3 (2019)

EfficientNet

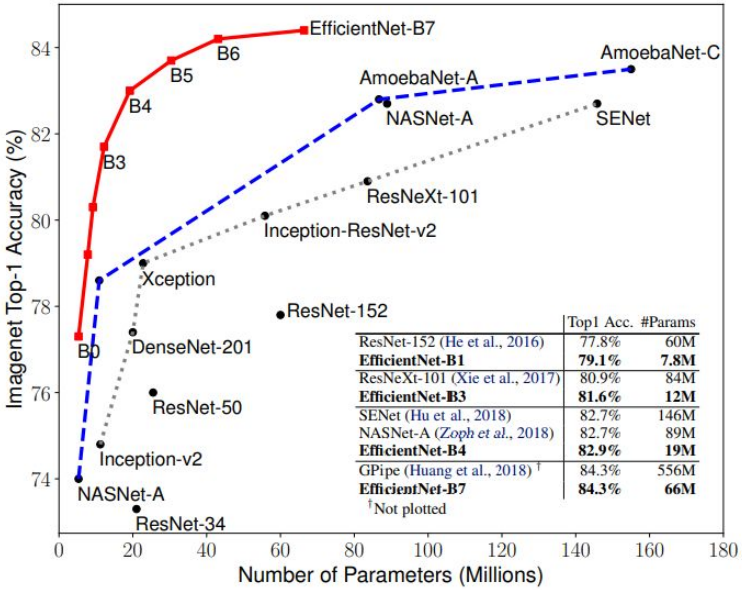
We can improve quality of the model by increasing resolution/depth/width of the model. What is the optimal balance between them?



$$\text{depth: } d = \alpha^\phi$$
$$\text{width: } w = \beta^\phi$$
$$\text{resolution: } r = \gamma^\phi$$
$$\text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$
$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks (2019)

EfficientNet



	Top1 Acc.	#Params
ResNet-152 (He et al., 2016)	77.8%	60M
EfficientNet-B1	**79.1%**	**7.8M**
ResNeXt-101 (Xie et al., 2017)	80.9%	84M
EfficientNet-B3	**81.6%**	**12M**
SENet (Hu et al., 2018)	82.7%	146M
NASNet-A (*Zoph et al.*, 2018)	82.7%	89M
EfficientNet-B4	**82.9%**	**19M**
GPipe (Huang et al., 2018) †	84.3%	556M
EfficientNet-B7	**84.3%**	**66M**

†Not plotted

EfficientNet problems

1. When large image resolution was used to train the large models, the training was slow.

2. In the early layers of the network architecture, depthwise convolution layers (MBConv) were slow. Depthwise convolutional layers generally have fewer parameters than regular convolutional layers, but the problem is that they cannot fully make use of modern accelerators.

3. Equal scaling was applied to the height, width, and image resolution to create the various EfficientNet models from B0 to B7. This equal scaling of all layers is not optimal.

EfficientNetV2

Adding a combination of MBConv and Fused-MBConv blocks

MBConv block often cannot fully make use of modern accelerators. Fused-MBConv layers can better utilize server/mobile accelerators.

NAS search to optimize Accuracy, Parameter Efficiency, and Training Efficiency

Intelligent Model Scaling

i. maximum image size was restricted to 480x480 pixels to reduce GPU/TPU memory usage, hence increasing training speed.
ii. more layers were added to later stages, to increase network capacity without increasing much runtime overhead.

Progressive Learning
The idea is very simple. In the earlier steps, the network was trained on small images and weak regularization. This allows the network to learn the features fast. Then the image sizes are gradually increased, and so are the regularizations.

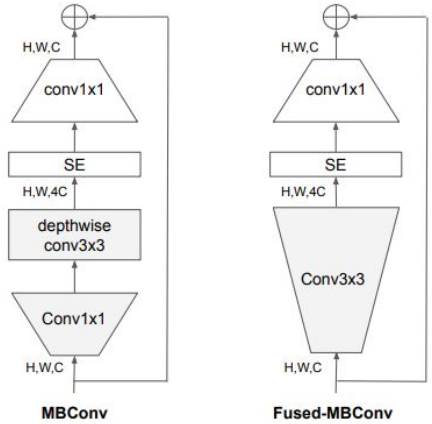


Figure 2. Structure of MBConv and Fused-MBConv.

EfficientNetV2: Smaller Models and Faster Training (2021)

ViT: Vision Transformer

https://nn.labml.ai/transformers/vit/index.html

ViT: Patching

How to adapt transformer for computer vision? First thing to deal with is to attention. If we consider one pixel as a token we have our attention matrix will have size (500x500) ^ 2 which is huge. So, the authors proposed to slice image on patches and work with them.



1) Slice



2) Unfold

```
from einops import rearrange

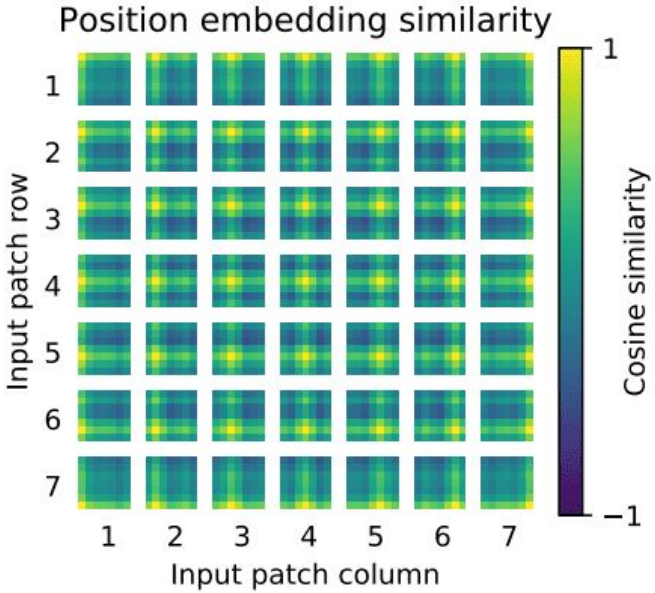
p = patch_size # P in maths

x_p = rearrange(img, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)', p1 = p, p2 = p)
```

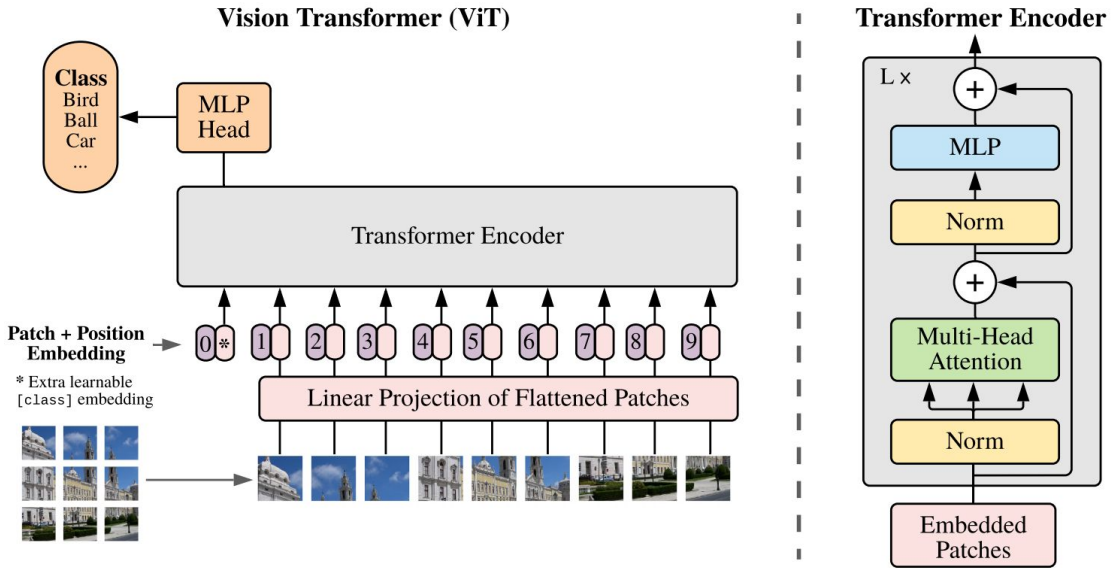
Code to implement this operation

ViT: Positional embeddings

As in transformers in NLP we have to add positional information to embeddings. But the images have 2D positional information (x, y). Authors tried 1D positional embeddings and 2D ones (concatenated row and column embeddings): both work with the same quality. And indeed the learned embeddings of neighbouring patches are similar.To fine-tune in higher resolutions, 2D interpolation of the pre-trained position embeddings is performed.

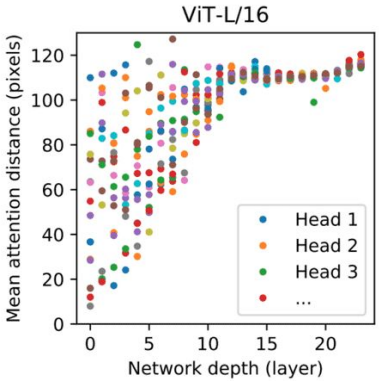


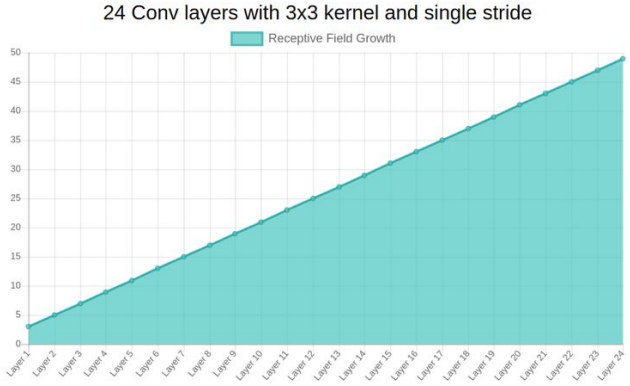
ViT: architecture

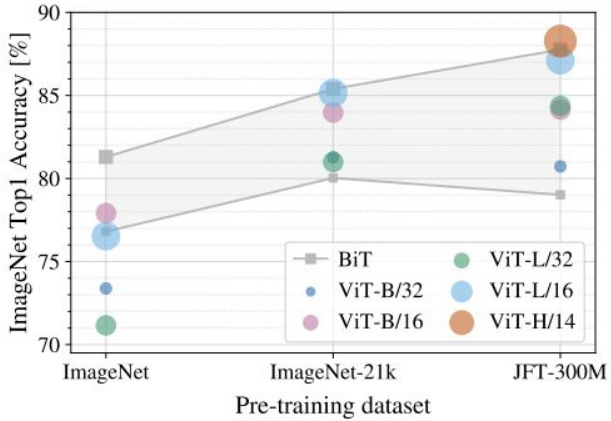


An image is worth 16x16 words: transformers for image recognition at scale (2021)

ViT: benefits







1) ViT has a global perception field in contrast of CNN. And from the figure we can see what model start to use this opportunity from the beginning

2) ViT continues to scale with data, CNN doesn't

Model Zoo: Where to find a model for your task





https://huggingface.co/models

https://huggingface.co/docs/timm/index



https://pytorch.org/vision/stable/models.html#classification

Recap

- SIFT
- AlexNet
- VGG
- Inception
- ResNet
- Xception
- MobileNet
- EfficientNet
- ViT

Quiz #7



https://forms.gle/b2AjDF1NVcTaWVso8